**REPUBLIQUE DU CAMEROUN**
*Paix – Travail – Patrie*
- . - . - . –
**UNIVERSITÉ DE YAOUNDÉ I**
**Faculté des Sciences**
*Département d'Informatique*
*B.P. 812 Yaoundé*




**REPUBLIC OF CAMEROON**
*Peace – Work – Fatherland*
- . - . - . –
**UNIVERSITY OF YAOUNDÉ I**
**Faculté des Sciences**
*Department of Computer Science*
*P.O.Box 812 Yaoundé*

# RAPPORT DE INF4067 : PRINCIPES SOLID

**FILIÈRE : INFORMATIQUE**
**NIVEAU : MASTER 1**
**ENCADREUR : Dr. MONTHE Valéry**

**NOMS:** ZABDJIO TSOPDJIO
**PRENOMS:** INA
**MATRICULE:** 23U2449

**ANNEE ACADEMIQUE 2026-2027**

## 1. Introduction

Ce rapport présente les modèles (diagrammes de classe) réalisés pour le projet INF4067. Le projet illustre, avec des exemples en Java, la notion de couplage et de cohésion ainsi que les cinq principes SOLID de la conception orientée objet : SRP, OCP, LSP, ISP et DIP.
Pour chaque notion, deux versions d'un même exemple ont été écrites : un **mauvais design**, qui montre le problème, et un **bon design**, qui le corrige. Chaque version est accompagnée de son diagramme de classe.

## 2. Code source

Le code complet du projet est disponible sur GitHub :
**Lien du dépôt :** https://github.com/inazabdjio/INF4067_SOLID

Il contient le dossier src (un package par notion), ainsi qu'un fichier README.md qui décrit le projet et explique comment exécuter chaque fichier.
J'ai dû modifier les noms de certaines classes pour ne pas créer de conflits.

## 3. Modèles (diagrammes de classe)

### 3.1 Couplage et cohésion\

#### 3.1.1 Mauvais design – CommandeBad.java

La classe **CommandeBad** calcule le total d'une commande et le sauvegarde dans MySQL. Elle crée directement un objet **MysqlConnection** : la cohésion est faible et le couplage est fort.

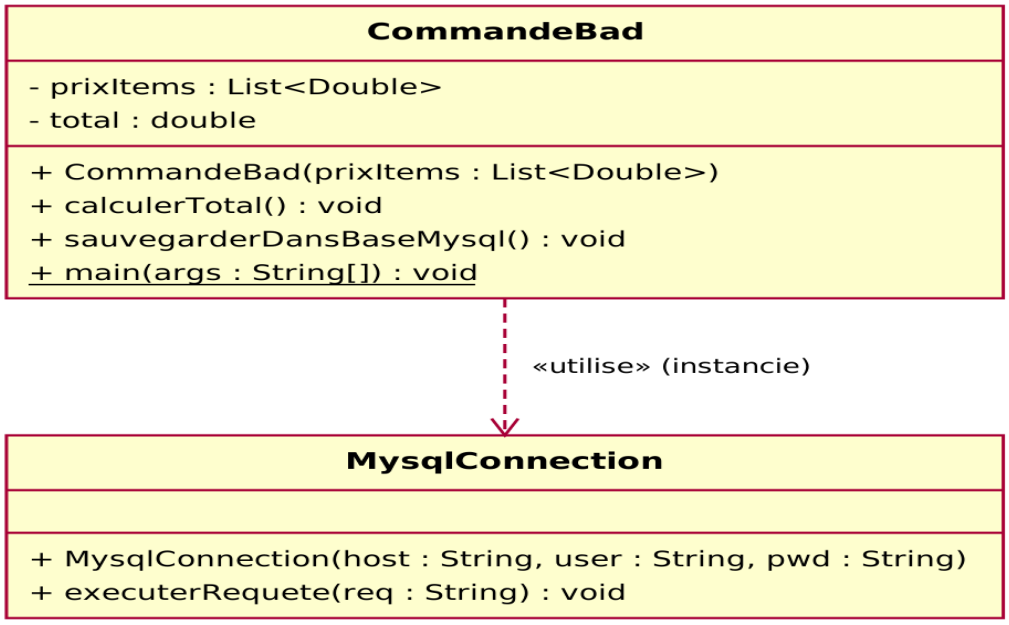



*Figure 1 – Diagramme de classe de **CommandeBad** (couplage fort)*

Ce que montre le diagramme **:** CommandeBad dépend directement de la classe concrète MysqlConnection, qu'elle instancie dans sauvegarderDansBaseMysql().

### 3.1.2 Bon design – CommandeGood.java

La classe Commande ne gère que les données et le calcul du total. La classe CommandeRepository s'occupe de la sauvegarde et reçoit une interface BaseDonnees par injection de dépendances.

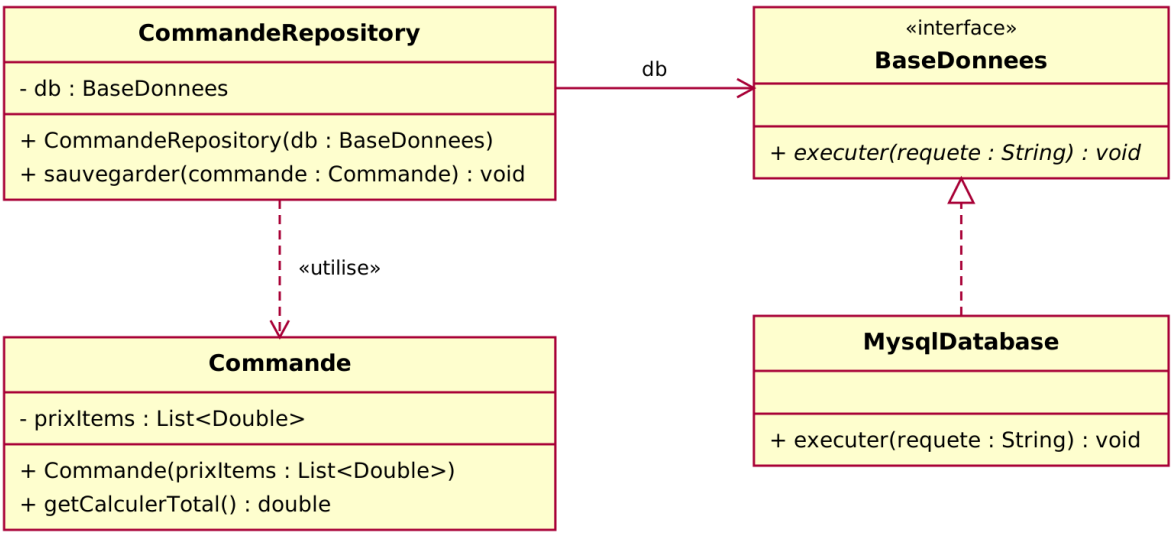



*Figure 2 – Diagramme de classe de **CommandeGood** (couplage faible)*

Ce que montre le diagramme : MysqlDatabase implémente l'interface BaseDonnees. CommandeRepository est associée à l'interface et non à la classe concrète, donc la base de données est interchangeable.

## 3.2 Responsabilité unique (SRP)

### 3.2.1 Mauvais design – BookBad.java

Une seule classe, BookBad, cumule quatre responsabilités : les données du livre, l'affichage, la sauvegarde en base et la logique d'emprunt.

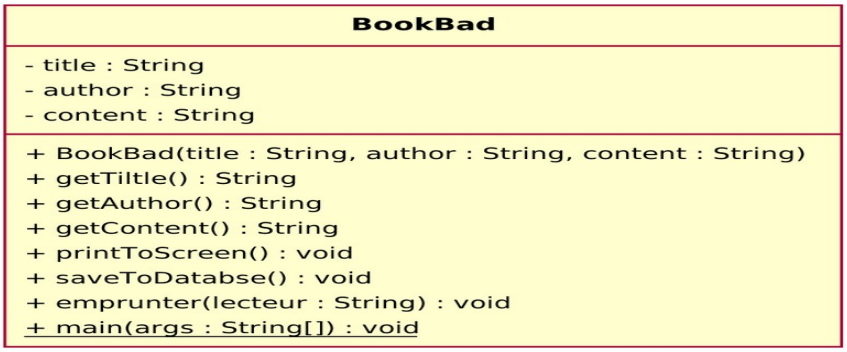



*Figure 3 – Diagramme de classe de **BookBad** (plusieurs responsabilités)*

Ce que montre le diagramme : une classe unique contient les getters, printToScreen, saveToDatabse et emprunter.

### 3.2.2 Bon design – BookGood.java

Chaque responsabilité est confiée à une classe : BookSRP (données), BookPrinter (affichage), BookSaver (sauvegarde) et BookBusinessLogic (logique métier).

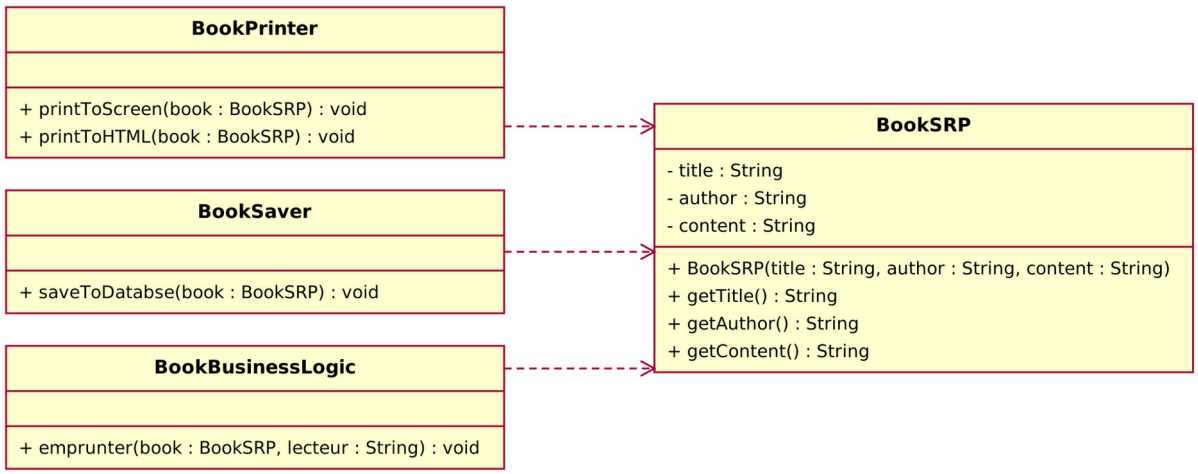



*Figure 4 – Diagramme de classe de **BookGood** (une classe par responsabilité)*

Ce que montre le diagramme : les trois classes de service dépendent de BookSRP, qu'elles reçoivent en paramètre de leurs méthodes.

## 3.3 Ouvert / fermé (OCP)

### 3.3.1 Mauvais design – AreaCalculatorBad.java

La méthode calculateArea(Object) enchaîne des tests instanceof. Pour ajouter une nouvelle forme, il faut modifier cette méthode.

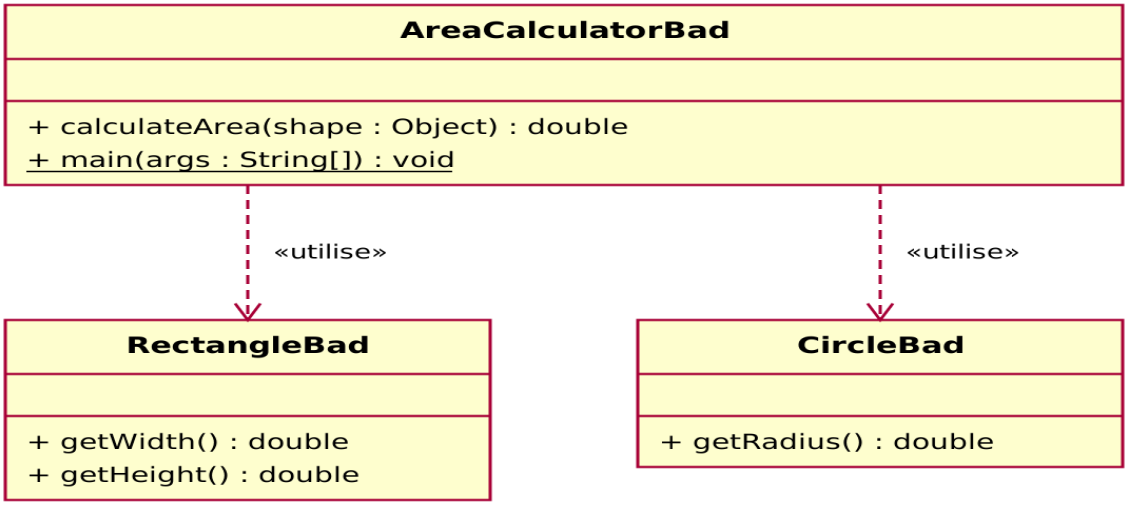



*Figure 5 – Diagramme de classe de **AreaCalculatorBad***

Ce que montre le diagramme : RectangleBad et CircleBad n'ont aucun type commun, et AreaCalculatorBad dépend de chacune d'elles.

### 3.3.2 Bon design – OCP.java

Une interface Shape est implémentée par Rectangle et Circle. AreaCalculator travaille avec Shape : pour ajouter une forme, il suffit de créer une nouvelle classe.

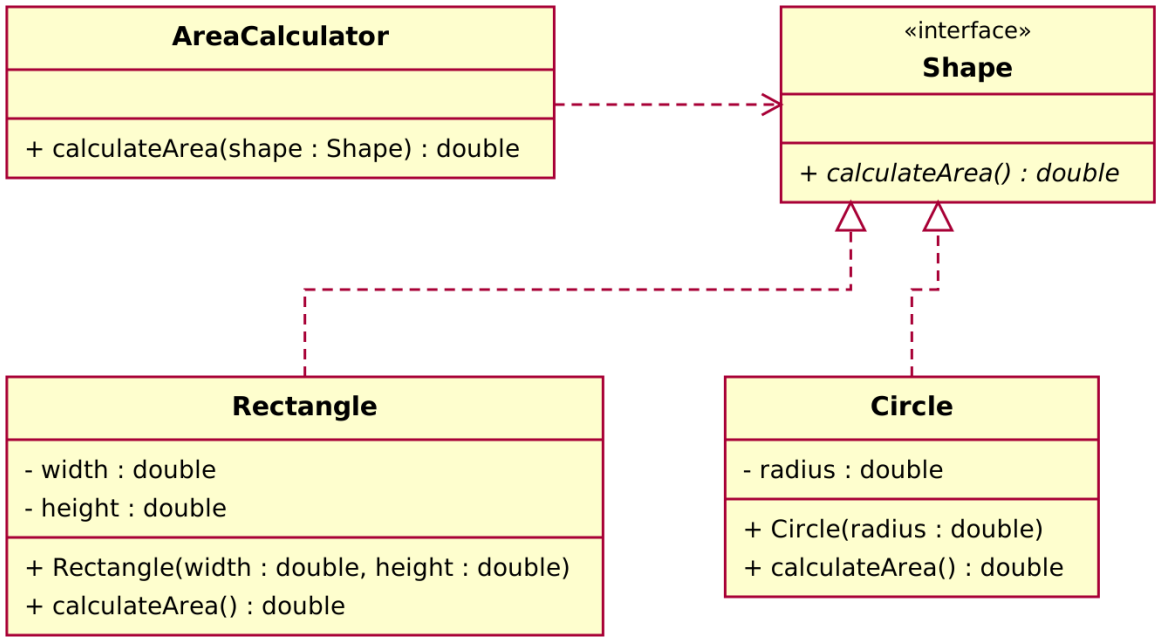



*Figure 6 – Diagramme de classe de **OCP (interface Shape)***

Ce que montre le diagramme : Rectangle et Circle implémentent Shape, et AreaCalculator dépend uniquement de cette interface.

### 3.3.3 Exemple complémentaire – Payment.java

La méthode processPayment de PaymentProcessor choisit le traitement avec une chaîne de if / else if sur le type de paiement (mobile, carte, PayPal, crypto). Chaque nouveau moyen de paiement oblige à modifier cette méthode, ce qui viole l'OCP.

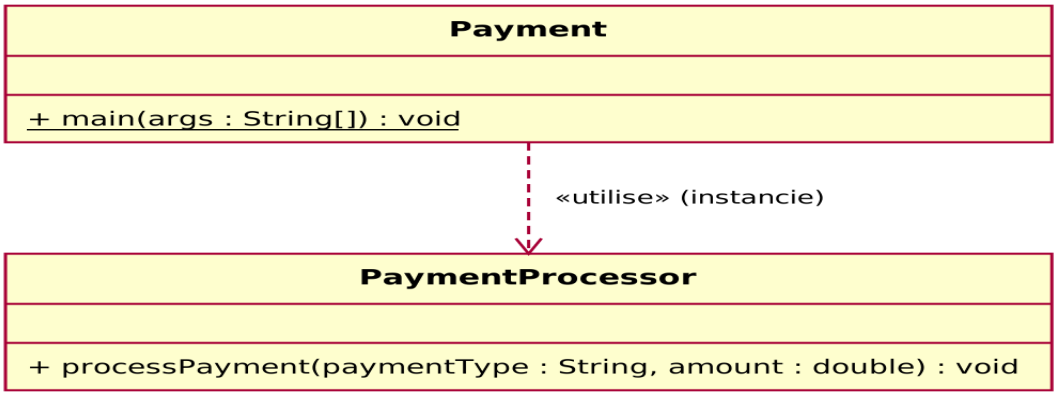


*Figure 7 – Diagramme de classe de **Payment et PaymentProcessor***

Ce que montre le diagramme : Payment instancie PaymentProcessor, qui ne contient qu'une seule méthode, processPayment. Le problème se trouve dans le code de cette méthode, pas dans la structure des classes.

### 3.4 Substitution de Liskov (LSP)

#### 3.4.1 Mauvais design – mainlspBad.java

SquareBad hérite de RectangleBad et redéfinit les setters pour modifier la largeur et la hauteur ensemble. Un carré utilisé à la place d'un rectangle donne une aire de 16 au lieu de 20.

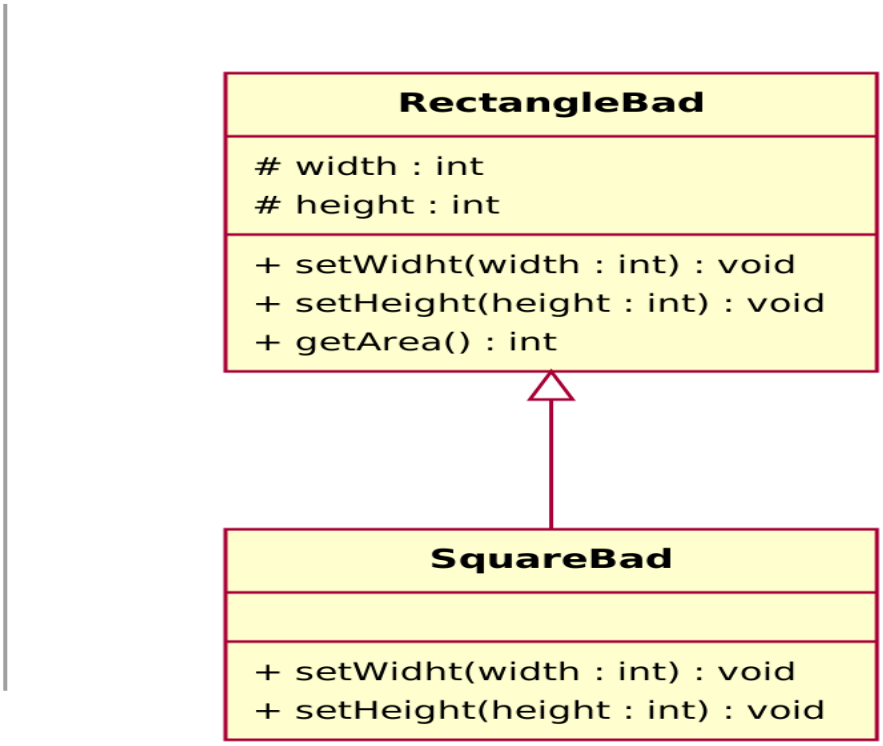



*Figure 8 – Diagramme de classe de **RectangleBad et SquareBad***

Ce que montre le diagramme : SquareBad hérite de RectangleBad et redéfinit setWidht et setHeight. Les attributs width et height sont protégés.

#### 3.4.2 Bon design – mainlspGood.java

Square et Rectangle n'héritent plus l'une de l'autre : elles implémentent chacune l'interface Shape et calculent leur propre aire.

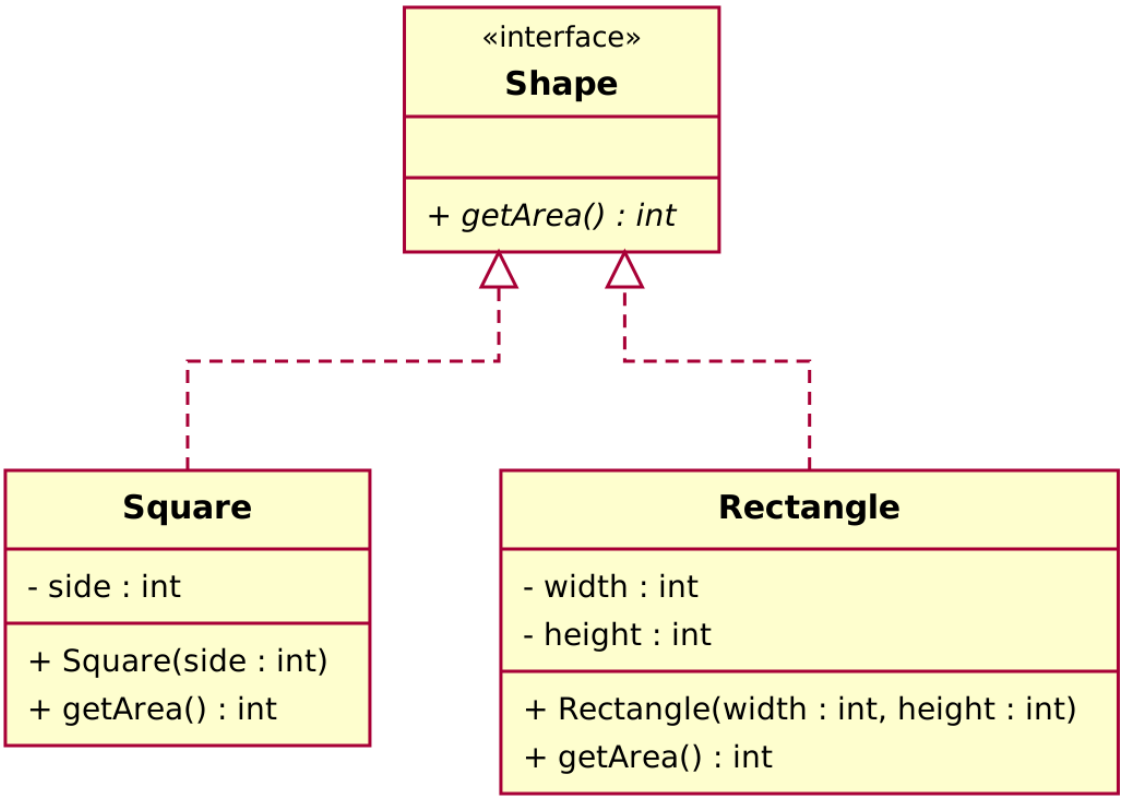



*Figure 9 – Diagramme de classe de **mainlspGood (interface Shape)***

Ce que montre le diagramme : Square et Rectangle implémentent Shape, avec leurs propres attributs (side, ou width et height).

### 3.5 Ségrégation des interfaces (ISP)

#### 3.5.1 Mauvais design – WorkerBad.java

L'interface WorkerBadInterface est trop large : elle impose work() et eat(). RobotWorkerBad doit implémenter eat() alors qu'un robot ne mange pas, ce qui lève une UnsupportedOperationException.

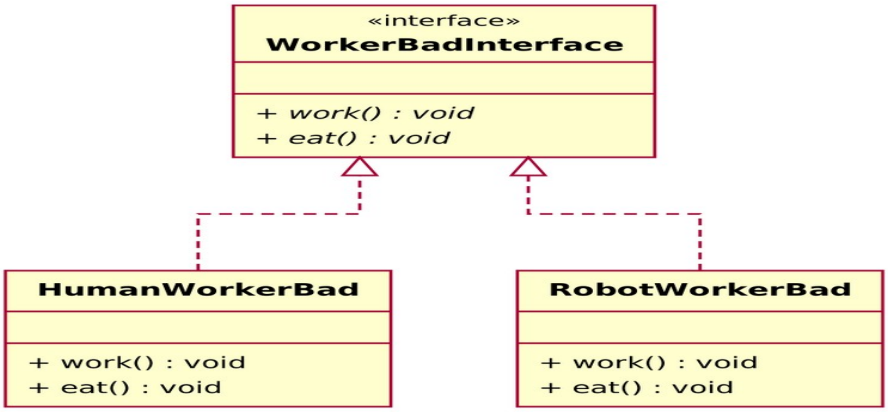


*Figure 10 – Diagramme de classe de **WorkerBad (interface trop large)***

Ce que montre le diagramme : HumanWorkerBad et RobotWorkerBad implémentent toutes les deux les deux méthodes de l'interface.

### 3.5.2 Bon design – WorkerGood.java

L'interface est découpée en Workable (work()) et Eatable (eat(), qui étend Workable). Chaque classe n'implémente que ce dont elle a besoin.

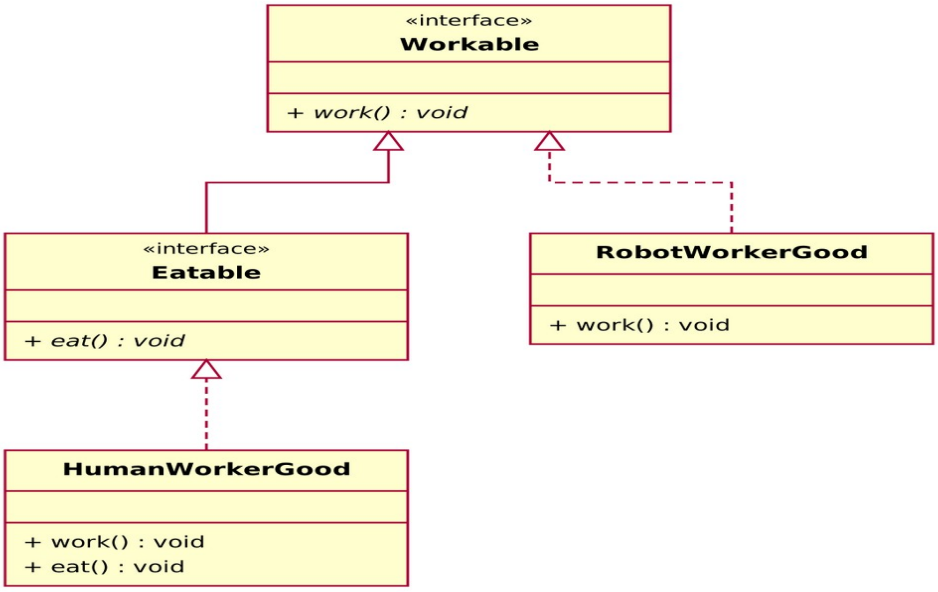



*Figure 11 – Diagramme de classe de **WorkerGood (interfaces séparées)***

Ce que montre le diagramme : Eatable étend Workable. HumanWorkerGood implémente Eatable, tandis que RobotWorkerGood implémente seulement Workable.

## 3.6 Inversion des dépendances (DIP)

### 3.6.1 Mauvais design – MainBad.java

La classe de haut niveau OrderProcessorBad crée directement une classe concrète de bas niveau, MySQLDatabaseBad.

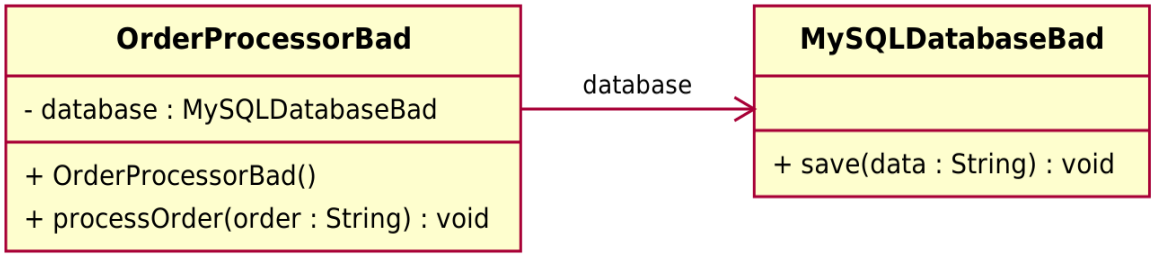



*Figure 12 – Diagramme de classe de OrderProcessorBad et MySQLDatabaseBad*

Ce que montre le diagramme : OrderProcessorBad possède un attribut database de type MySQLDatabaseBad, une classe concrète, sans interface entre les deux.

### 3.6.2 Bon design – MainGood.java

OrderProcessor dépend de l'interface Database, qu'elle reçoit par son constructeur. MySQLDatabase et MongoDBDatabase l'implémentent : on peut changer de base de données sans modifier OrderProcessor.

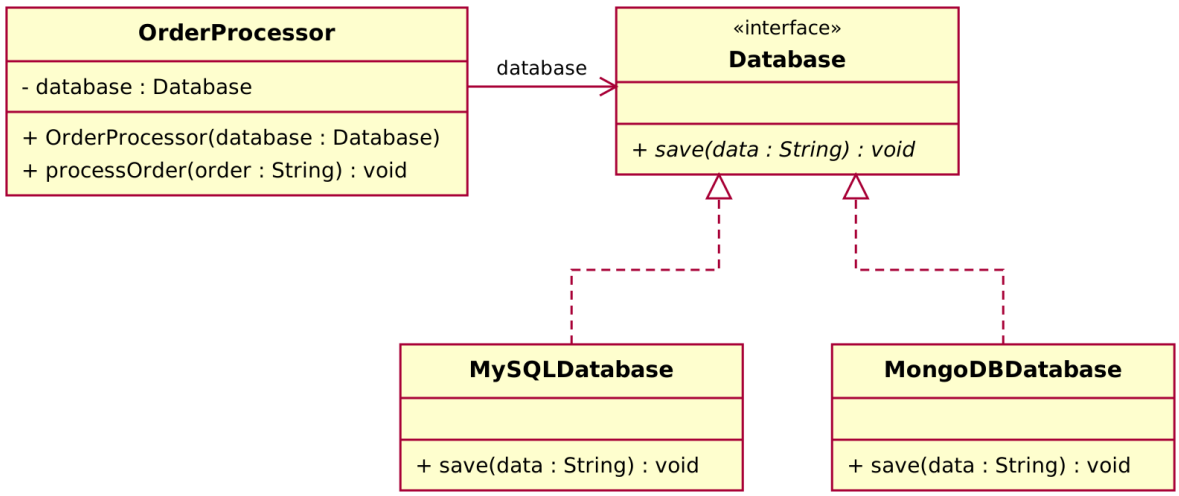



*Figure 13 – Diagramme de classe de MainGood (dépendance vers l'abstraction)*

Ce que montre le diagramme : OrderProcessor est associée à l'interface Database, implémentée par MySQLDatabase et MongoDBDatabase.

## 5. Conclusion

Ce travail a permis de comparer, pour chaque notion, un mauvais design et un bon design. Les diagrammes de classe montrent comment l'utilisation d'interfaces, la séparation des responsabilités et l'injection de dépendances réduisent le couplage et rendent le code plus facile à faire évoluer.